

ADAMOPS: An Automated End-to-End MLOPS Framework - II REPORT

**A PROJECT REPORT ON 6CS1991 – DESIGN
PROJECT - II**

Submitted by

Deepesh Kumar Jha	2023BCSE07AED472
Ayush Mishra A	2023BCSE07AED459
Abhishek V Ammanagi	2023BCSE07AED500
Mohd Mustafa	2023BCSE07AED397

in partial fulfillment for the award of the degree of

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING (AIML)**

Under the Supervision of
Dr. Viji C



ALLIANCE SCHOOL OF ADVANCED COMPUTING

**ALLIANCE UNIVERSITY: BANGALORE
562106**

APRIL - 2026

ADAMOPS :An Automated End-to-End MLOPS Framework

A PROJECT REPORT ON 6CS1991 – DESIGN PROJECT-II

Submitted by

Deepesh Kumar Jha	2023BCSE07AED472
Ayush Mishra A	2023BCSE07AED459
Abhishek V Ammanagi	2023BCSE07AED500
Mohd Mustafa	2023BCSE07AED397

In partial fulfilment for the award of the degree of

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING (AIML)**

Under the Supervision of

Dr. Viji C
Associate Professor
Department of CSE



Department of Computer Science and Engineering

ALLIANCE SCHOOL OF ADVANCED COMPUTING
Alliance University - Bangalore-562106
APRIL- 2026

**Department of Computer Science and Engineering / Information
Technology
Alliance School of Advanced Computing
Alliance University
Chikkahagade Cross, Chandapura-Anekal Main Road, Bangalore-562106**



CERTIFICATE

This is to certify that the Design Project – II work entitled “**ADAMOPS :An Automated End-to-End MLOPS Framework**” is the Bonafide work done by **Mr. Deepesh Jha (2023BCSE07AED472) Mr.Ayush Mishra A(2023BCSE07AED459) Mr.Abhishek V Ammanagi (2023BCSE07AED500) Mr.Mohd Mustafa (2023BCSE07AED397)** submitted in partial fulfillment of the requirements for the Award of the Degree **Bachelor of Technology in Computer Science and Engineering** during the year 2023- 2027.

**Supervisor name
Dr Viji C**

**HOD-name
Dr Ramalakshmi K**

External Examiners:

- | | | |
|----|--------------|------------------|
| 1. | Name: | Signature |
| 2. | Name: | Signature |

**Department of Computer Science and Engineering
Alliance School of Advanced Computing**

Alliance University
Chikkahagade Cross,
Chandapura-Anekal Main Road,
Bangalore-562106



CERTIFICATE

This is to certify that the Design Project – II work entitled “**ADAMOPS :An Automated End-to-End MLOPS Framework**” is the Bonafide work done by **Mr. Deepesh Jha(2023BCSE07AED472), Mr. Ayush Mishra A (2023BCSE07AED459),Mr. Abhishek V Ammanagi (2023BCSE07AED500), Mr. Mohd Mustafa (2023BCSE07AED397)**submitted in partial fulfilment of the requirements for the award of the degree **Bachelor of Technology in Computer Science and Engineering** during the year 2023-2027.

Dr. Viji C
SUPERVISOR

Dr. Ramalakshmi K
HOD

External Examiners: 1. Name: _____ Signature: _____

2. Name: _____ Signature: _____



ALLIANCE
UNIVERSITY

NAAC
GRADE A+
ACCREDITED UNIVERSITY

Declaration

This is to declare that the report titled “**ADAMOPS :An Automated End-to-End MLOPS Framework**” has been made for the partial fulfilment of the Course Bachelor of Technology in Computer Science and Engineering , under the Supervision of **Dr. Viji C.** We confirm that this report truly represents our work undertaken as a part of our project work. This work is not a replication of work done previously by any other person. We also confirm that the contents of the report and the views contained therein have been discussed and deliberated with the faculty guide.

NAME	REG. NO	SIGNATURE
Deepesh Kumar Jha	2023BCSE07AED472	
Abhishek V Ammanagi	2023BCSE07AED500	
Ayush Mishra A	2023BCSE07AED459	
Mohd Mustafa	2023BCSE07AED397	



ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of the task would be put incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all the efforts with success.

We are very thankful to our Guide **Dr. Viji C** , Chief Coordinator **Dr. M. Selvam** and Project Coordinator **Dr. Asha Kurian** , Department of Computer Science and Engineering (Specialization) / Department of Computer Science and Engineering / Department of Information Technology for his / her sustained inspiring guidance and cooperation throughout the process of this project. His / Her wise counsel and valuable suggestions are invaluable.

We would like to thank **Dr. Ramalakshmi K** Head of the department, and **Dr. Vullikant Vivek**, Deputy Director, for their encouragement and cooperation at various levels of Project.

We avail this opportunity to express my deep sense of gratitude and hearty thanks to the Management of Alliance University, for providing world class infrastructure, congenial atmosphere and encouragement.

We express my deep sense of gratitude and thanks to the teaching and non-teaching staff at our department who stood with me during the project and helped me to make it a successful venture.

We place highest regards to my parents, my friends and well-wishers who helped a lot in making the report of this project.

Deepesh kumar jha
Ayush mishra A
Abhishek V Ammanagi
Mohd Mustafa

APPENDIX 3

TABLE OF CONTENTS			
CHAPTER NO.	TITLE		PAGE NO.
1	INTRODUCTION		1
	1.1	MLOps Introduction	1
	1.2	State of the Art (Survey of Literature)	2-3
2	LITERATURE REVIEW		4
3	OBSERVATIONS FROM THE LITERATURE REVIEW		5
4	LIMITATIONS OF THE LITERATURE REVIEW		6
5	PROBLEM DEFINITION AND PROPOSED RESEARCH		
	5.1	PROBLEM STATEMENT	7
	5.2	PROPOSED RESEARCH	8
	5.3	HARDWARE REQUIREMENTS	9
	5.4	SOFTWARE REQUIREMENTS	10
6	METHODOLOGY		11-13
7	MODULE DESCRIPTION		14-16
8	EXPERIMENTATION		17-20
9	RESULTS AND DISCUSSION		21
10	CONCLUSION AND FUTURE DIRECTIONS		22
11	REFERENCES		23
12	APPENDIX: PLAGIARISM REPORT		24
13	APPENDIX: PUBLICATION SUBMISSION PAGE		25
14	APPENDIX: SOURCE CODE		26-30

CHAPTER 1

INTRODUCTION:

1.1 MLOps Introduction

The core theme of this Design Project-II is the development and open-source release of ADAMOPS – a unified, low-code Python-based MLOps framework that automates the entire machine learning lifecycle from data ingestion to production deployment and monitoring. fundamental principle underlying ADAMOPS is democratization of MLOps through intelligent abstraction. Traditional MLOps workflows are highly fragmented: data scientists and ML engineers spend only about 10 % of their time on actual model building and 90 % on repetitive, error-prone tasks such as data versioning, experiment tracking, pipeline orchestration, quality validation, containerization, and cloud deployment. ADAMOPS solves this by providing a single high-level Python interface (simple function calls and configuration objects) that sits on top of best-in-class open-source tools (DVC, MLflow, Great Expectations, Docker, Terraform, etc.)

The framework follows four core “Connect” modules – Data Connect, Developer Connect, Testing Connect, and Deployment Connect – that work together as a seamless pipeline. This design ensures that even developers with entry-level ML skills can move from raw data to a production-ready, containerized, and cloud-deployable model in fewer than a dozen lines of code while retaining full manual control when needed. By packaging ADAMOPS as a reusable Python library now available on PyPI (pip install adamops), the project transforms MLOps from a specialist-only discipline into an accessible, reproducible, and scalable engineering practice.

1.2 State of the Art (Survey of Literature)

The literature on MLOps consistently highlights the same pain points that ADAMOPS addresses. Fragmentation and tool sprawl: Multiple systematic reviews (Zarour et al., 2025; Berberi et al., 2025; Hanchuk & Semerikov, 2025) show that existing MLOps platforms are either vendor-locked (heavy AWS dependency), cover only isolated stages of the lifecycle, or require deep DevOps expertise. High barrier to entry: Papers such as Singla (2023), Symeonidis et al. (2022), and John et al. (2025) emphasize that successful deployment demands a rare combination of ML, cloud, and DevOps skills, leading to 87 % of ML models never reaching production. Lack of end-to-end automation: Edge MLOps frameworks (Raj et al., 2021) and AWS-centric pipelines (Jana, 2025) provide partial automation but ignore DataOps, reproducibility, or lightweight open-source alternatives. AutoML tools remain “black-box” and rarely

integrate with CI/CD or monitoring (Zhou et al., 2020). Emerging maturity models: Recent works (Woźniak et al., 2025; Bayram & Ahmed, 2025; Li et al., 2025) propose maturity frameworks and trustworthiness metrics, yet no single open-source solution offers a unified abstraction layer with agentic Docker-file generation and cloud-agnostic deployment. ADAMOPS differentiates itself by integrating these best practices into one cohesive, open-source Python package that is both lightweight for individual developers and scalable for enterprise use.

1.3 What Has Been Done in This Work (Design Project-II)

Building upon the design, literature review, and proof-of-concept architecture completed in Design Project-I (November 2025), the present Design Project-II focused on full implementation, rigorous testing, documentation, and open-source distribution. Complete modular implementation of all four Connect modules with production-grade code. Seamless integration of DVC, MLflow, Great Expectations, Docker, and Terraform behind a clean Python API. Development of the intelligent Auto-DockerFile Agent that scans the user project and generates correct, optimized Dockerfiles automatically. End-to-end workflow validation on real ML projects (classification and regression use-cases) demonstrating drastic reduction in code volume and setup time. Packaging and publishing of ADAMOPS as an official Python package on PyPI (adamops), enabling one-command installation and version-controlled updates. Comprehensive unit, integration, and pipeline tests using GitHub Actions CI/CD. Detailed user documentation, example notebooks, and a public GitHub repository for community contributions. The outcome is a ready-to-use, open-source MLOps framework that any developer can install via pip and start using immediately, fulfilling the original vision of making production-grade ML accessible without requiring a dedicated MLOps team.

1.4 Chapter-wise Summary

Chapter 1: Introduction (present chapter) – states the problem, principle, literature survey, and overview of work done.

Chapter 2: Literature Review & Gap Analysis – expanded critical review of recent MLOps papers and clear identification of research gaps addressed by ADAMOPS.

Chapter 3: System Architecture and Detailed Design – complete UML diagrams, module-wise architecture, and internal workflow of each Connect.

Chapter 4: Implementation Details – code structure, key algorithms (including the Auto-DockerFile agent), integration with external tools, and PyPI packaging process.

Chapter 5: Testing, Validation & Performance Evaluation – test cases, metrics, comparative analysis with manual MLOps pipelines, and real-project results.

Chapter 6: Deployment, User Guide & Case Studies – step-by-step usage, cloud deployment examples (AWS/Azure/local), and live case studies.

Chapter 7: Conclusion, Limitations & Future Enhancements – summary of achievements, current limitations, and roadmap for v2.0 (Kubeflow integration, advanced monitoring, etc.).

CHAPTER 2

REVIEW OF LITERATURE

S.No	Author Name, Journal, Publication Year	Research Methodology	Performance Metrics	Advantages	Disadvantages
1	Zarour, Mohammad, Hamza Alzabut, and Khaled Alsarayah. "MLOps best practices, challenges and maturity models: A systematic literature review." <i>Information and Software Technology</i> (2025): 107733.	Introduces a structured approach to developing complete MLOps solutions on AWS by seamlessly combining multiple cloud services.	The outcomes of this approach were encouraging. The platform recorded roughly a 22% uplift in typical purchase size and about an 18% boost in overall customer interaction.	Delivers a robust, scalable, and fully integrated pipeline for automating machine-learning processes on AWS, enhancing operational productivity while reducing the need for human oversight.	Does not include any data-driven analysis or forecasting techniques, and its scope is limited to utilizing AWS-native services only.
2	Berberi, Lisana, et al. "Machine learning operations landscape: platforms and tools." <i>Artificial Intelligence Review</i> 58.6 (2025): 167.	Comprehensive review of the MLOps platforms and tools landscape, covering both open-source and closed-source solutions with detailed categorization	The paper focuses on landscape analysis rather than specific quantitative performance metrics.	Provides an inclusive and detailed overview of over 90 MLOps tools and platforms, highlighting capabilities, new trends, and criteria for evaluation	Lacks detailed empirical evaluations of individual platforms; it serves more as a broad qualitative survey of available options.
3	Raj, Emmanuel, et al. "Edge mlops: An automation framework for aiot applications." 2021 IEEE International Conference on Cloud Engineering (IC2E). IEEE, 2021.	Proposes a systematic framework for building end-to-end MLOps tools on AWS by integrating various services	The results of this implementation were positive. The e-commerce company observed a 20% increase in average order value and a 15% rise in user engagement.	Provides a comprehensive and scalable end-to-end solution for automating ML workflows on AWS. It improves operational efficiency and minimizes manual intervention.	Lacks empirical data analysis and predictive modeling. And only provides services that are available on AWS.
4	Singla, Amandeep. "Machine learning operations (mlops): Challenges and strategies." <i>Journal of Knowledge Learning and Science Technology</i> ISSN: 2959-6386 (online) 2.3 (2023): 333-340.	This abstract discusses the importance of MLOps in managing the complete ML lifecycle, outlines technical, organizational, and cultural challenges.	The proposed MLOps practices improve scalability, deployment consistency, reproducibility, and reliability of ML	MLOps improves scalability, reliability, and reproducibility of ML systems, enhances collaboration between teams, enables faster and safer deployments through automation, and ensures consistent model performance in production.	Implementing MLOps requires significant initial effort, tooling complexity, and skill development, may face cultural resistance within organizations, and can increase operational overhead for smaller teams or simpler ML projects.
5	Symeonidis, Georgios, et al. "Mlops-definitions, tools and challenges." 2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC). IEEE, 2022.	This paper provides a focused overview of MLOps, defining its components, tools, challenges, and emerging trends.	It highlights the integration of MLOps with AutoML, explainability, and sustainability to build efficient, robust, and production-ready ML systems.	Proposes AutoML as an concept to incorporate in various tools.	AutoML is still considered as a blackbox tool due to the complexity involved
6	Zhou, Yue, Yue Yu, and Bo Ding. "Towards mlops: A case study of ml pipeline platform." 2020 International conference on artificial intelligence and computer engineering (ICAICE). IEEE, 2020.	The Study evaluates Devops and MLOPS framework as a tool and as the framework	The performance analysis reveals uneven resource utilization across pipeline stages, highlights GPU underutilization as a key bottleneck.	This work offers practical insights into building an ML platform using CI/CD and Kubeflow, systematically analyzes time and resource consumption.	The evaluation is limited to specific tools and configurations, performance results may not generalize to all ML platforms or workloads.

CHAPTER 3

OBSERVATIONS FROM LITERATURE REVIEW

- Strong pipeline automation exists in current MLOps solutions, yet tools remain highly fragmented and frequently vendor-locked, forcing developers to manage multiple disconnected platforms.
- Most existing frameworks cover only isolated stages of the ML lifecycle (data handling, training, or deployment) and rarely provide seamless end-to-end integration.
- AutoML tools, while powerful, are often treated as black-box solutions and lack native integration with full MLOps pipelines, monitoring, versioning, or CI/CD workflows.
- Edge and AIoT applications require lightweight, decentralized MLOps frameworks, but the majority of available solutions are cloud-centric and resource-heavy.
- Current MLOps platforms demand significant technical expertise in ML engineering, DevOps, cloud infrastructure, and containerization, creating a steep learning curve that limits accessibility for individual developers and small teams.
- Open-source tools, although cost-effective, suffer from poor interoperability, complex configuration requirements, and the absence of a unified abstraction layer.
- Enterprise-grade platforms prioritize large-scale performance and scalability but overlook usability, simplicity, and the needs of non-expert users or resource-constrained environments.
- Open-source tools, although cost-effective, suffer from poor interoperability, complex configuration requirements, and the absence of a unified abstraction layer.
- Enterprise-grade platforms prioritize large-scale performance and scalability but overlook usability, simplicity, and the needs of non-expert users or resource-constrained environments.
- High dependency on proprietary cloud services (especially AWS) leads to vendor lock-in, increased costs, and reduced flexibility for organizations seeking open-source alternatives.
- DataOps aspects such as automated preprocessing, Auto-EDA, data validation, and versioning are inconsistently addressed or entirely missing in most reviewed frameworks.
- Reproducibility, model drift detection, continuous monitoring, and automated containerization (Dockerfile generation) are mentioned as challenges but are not comprehensively solved in a single open-source package.

CHAPTER 4

LIMITATIONS OF LITERATURE REVIEW

- Most reviewed frameworks are fragmented and cover only specific stages of the ML lifecycle, with no single solution providing true end-to-end automation from data ingestion to production deployment.
- Heavy reliance on proprietary cloud platforms (especially AWS) leads to vendor lock-in, high costs, and limited flexibility for users preferring open-source or multi-cloud environments.
- Existing tools demand advanced expertise in DevOps, cloud infrastructure, containerization, and MLOps, making them inaccessible to individual developers, students, and small teams with limited experience.
- DataOps features such as automated preprocessing, Auto-EDA, data validation, and versioning are either missing or inconsistently implemented across the studied frameworks.
- AutoML and pipeline automation solutions remain “black-box” in nature and lack seamless integration with experiment tracking, model versioning, monitoring, or CI/CD pipelines.
- Edge and AIoT applications are poorly supported; most frameworks are cloud-centric and resource-heavy, offering little support for lightweight, decentralized deployments.
- Reproducibility, model drift detection, continuous monitoring, and automated containerization (especially intelligent Dockerfile generation) are frequently identified as challenges but are rarely solved comprehensively in one package.
- Open-source tools suffer from poor interoperability, complex configuration requirements, and steep learning curves due to the absence of a unified high-level abstraction layer.
- Very few studies provide empirical performance metrics or real-world validation of complete end-to-end workflows; most focus on theoretical architectures or narrow case studies.
- Maturity models and best-practice guidelines exist in recent literature, yet no practical, ready-to-use open-source Python package (installable via pip) combines all critical MLOps components into a simple, low-code interface.

CHAPTER 5

a. PROBLEM STATEMENT

The absence of a unified, low-code, open-source MLOps framework results in highly fragmented ML workflows, steep technical barriers, vendor lock-in, and an alarmingly high failure rate (approximately 87 %) of machine learning models never reaching production.

b. PROPOSED RESEARCH

The proposed research work aims to design, develop, and open-source a complete end-to-end MLOps framework named **ADAMOPS** that automates the entire machine learning lifecycle from raw data ingestion to production deployment and monitoring using a simple, high-level Python interface.

ADAMOPS acts as a single unified abstraction layer built on top of proven open-source tools, eliminating the need for developers to manually integrate multiple fragmented platforms. The framework is organized into four core modular components called “**Connects**”

- **Data Connect:** Handles automated data preprocessing, Auto-EDA report generation, data validation using Great Expectations, and Data Versioning with DVC.
- **Developer Connect:** Provides seamless experiment tracking with MLflow, model versioning, hyperparameter optimization, and automatic registration of the best models in the MLflow Model Registry.
- **Testing Connect:** Ensures pipeline reliability through component-level testing and I/O shape validation to prevent runtime errors in production.
- **Deployment Connect:** Automates containerization using Docker with an intelligent Auto-DockerFile Agent that scans the project structure and dynamically generates optimized Dockerfiles, followed by infrastructure provisioning using Terraform for cloud-agnostic deployment.

The entire workflow is orchestrated through minimal Python code (typically under a dozen lines), making MLOps accessible to developers with entry-level ML knowledge while still offering full manual control when required. The framework has been fully implemented, rigorously tested,

packaged as a production-ready Python library, and published on **PyPI** (pip install adamops), enabling one-command installation and immediate use.

The proposed solution is cloud-agnostic (supports AWS, Azure, or local deployment), emphasizes reproducibility, data quality, model drift readiness, and CI/CD integration via GitHub Actions. By addressing the critical gaps identified in the literature fragmentation, high expertise requirements, and incomplete automation ADAMOPS democratizes MLOps, significantly reduces development time, and bridges the gap between model building and real-world production deployment.

c. HARDWARE REQUIREMENTS

The ADAMOPS framework is designed to run efficiently on standard developer machines. The minimum and recommended hardware specifications are as follows:

- **Processor:** 4-core CPU (Intel Core i5 / AMD Ryzen 5 or equivalent) or higher
- **RAM:** 8 GB (16 GB or higher recommended for running multiple containers and large models)
- **Storage:** 256 GB SSD (minimum 2–3 GB free space required; higher capacity recommended for storing datasets, models, and Docker images)

d. SOFTWARE REQUIREMENTS

- **Operating System:** Cross-platform support – Ubuntu 20.04+ / macOS 11.0+ / Windows 10/11 (with WSL2 for Docker)
- **Programming Language:** Python 3.10+
- **Package Management:** pip and virtual environment (venv or conda)
- **Core MLOps Tools:**
 - DVC (Data Version Control)
 - MLflow (Experiment Tracking & Model Registry)
 - Great Expectations (Data Validation)
- **Deployment & Infrastructure:**
 - Docker & Docker Compose (Containerization)
 - Terraform (Infrastructure as Code)
- **Development & Automation:**

- Git & GitHub (Version Control & CI/CD)
- GitHub Actions (Automated testing and pipeline validation)

CHAPTER 6: METHODOLOGY

The methodology used to build ADAMOPS follows a simple but effective approach focused on continuous improvement and practical usability. Instead of trying to build everything at once, the system was developed step by step using an iterative process. Each part was designed, tested, improved, and then connected with the rest of the system. This made sure that errors were caught early and the system stayed stable as it grew. At its core, ADAMOPS is built as a Python library, so it is easy to install and use. The design is based on a layered architecture, where each layer handles a specific task like data processing, experiment tracking, pipeline creation, and deployment. This separation keeps the system organized and scalable. One key idea behind the methodology is automation. Once the user provides data, the system takes over tasks like cleaning, preprocessing, model selection, and pipeline building without requiring manual coding. At the same time, it integrates tools like MLflow, DVC, and Git in the background, so users get full tracking and version control without extra effort. The system was also tested thoroughly using unit tests, integration tests, and automated CI/CD pipelines to ensure reliability. Overall, the methodology focuses on reducing complexity, minimizing manual work, and making machine learning workflows faster, more consistent, and easier to use in real-world scenarios.

CHAPTER 7: MODULES AND COMPONENTS

7.1 Data Module

The Data Module handles everything related to input data and makes sure it is clean and ready to use. It supports multiple sources like CSV, Excel, JSON, SQL databases, and even APIs, so users are not limited by format. Once data is loaded, the system checks for common issues such as missing values, duplicate rows, and incorrect data types. After validation, preprocessing steps are applied automatically, including filling missing values, removing outliers, and cleaning text if needed. It also performs feature engineering by encoding categorical data, scaling numerical values, selecting important features, and sometimes even creating new ones. Finally, the module splits the data into training and testing sets using methods like standard splits, K-Fold cross-validation, or time-series splits. This ensures the data is fully prepared without requiring manual effort.

7.2 Model Module

The Model Module provides a wide range of machine learning algorithms to handle different types of problems. It includes regression models like Ridge, Lasso, XGBoost, and LightGBM, as well as classification models such as decision trees, Naive Bayes, KNN, and boosting methods. For unsupervised tasks, it supports clustering techniques like K-Means, DBSCAN, and Gaussian Mixture Models. The module also includes ensemble methods like stacking, blending, and voting to improve performance. One of its key features is AutoML, where the system automatically selects the best model and tunes its parameters using different optimization techniques. This reduces the need for manual experimentation.

7.3 Experiment Tracking

The Experiment Tracking module ensures that every run is recorded properly. It logs important details such as parameters used, performance metrics, and output artifacts. All this information is stored in MLflow, making it easy to compare experiments and track progress over time. When a model performs well, it can be promoted to a central model registry, where it is stored and managed for production use. This removes the need for manual tracking and reduces confusion.

7.4 Evaluation Module

Once a model is trained, the Evaluation Module checks how well it performs. It calculates relevant metrics depending on the task, such as accuracy, precision, recall, or error scores. It also generates visual outputs like confusion matrices, ROC curves, and feature importance plots. To make models more understandable, it uses explainability tools like SHAP and LIME. In addition, the module automatically generates detailed reports in HTML or PDF format, making results easy to interpret and share.

7.5 Deployment Module

The Deployment Module is responsible for taking trained models into production. It allows models to be exported in different formats such as ONNX, PMML, TensorFlow Lite, or CoreML, depending on the target environment. It can also create APIs using frameworks like FastAPI, Flask, or Streamlit, making the model accessible for real-time use. The module supports containerization using Docker and orchestration through Kubernetes. It also provides one-click deployment options for cloud platforms like AWS, GCP, and Azure, simplifying the entire deployment process.

7.6 Pipeline and Abstraction Layer

The Pipeline and Abstraction Layer acts as the core brain of ADAMOPS. It connects all other modules and turns user input into a complete machine learning workflow. The user only needs to describe the task, and the system automatically builds and executes the pipeline step by step. This design allows even non-technical users to work with machine learning systems without needing deep expertise. The system handles all decisions internally, making the entire process smooth and efficient.

CHAPTER 8: EXPERIMENTS

The experiments were designed to check whether ADAMOPS actually works in real conditions, not just in theory. We tested it on a mix of well-known datasets like Iris, Boston Housing, Titanic, Wine Quality, and Customer Churn, along with some custom datasets. The goal was simple see if the system stays stable across different types of data and does not break when conditions change. This helped us understand how reliable the framework is when used in different scenarios.

For model development, the system handled everything on its own. It first looked at the data and figured out what kind of problem it was, like classification or regression. Then it selected suitable models automatically. It started with basic hyperparameters and later improved performance using ensemble methods like stacking and blending. This removed the need for manual trial and error, which usually takes a lot of time.

Training was fully automated and required no manual setup. The system used cross-validation techniques to ensure the model learns properly and does not overfit. It also used parallel processing tools like Dask and Joblib to speed up computation, which is important when working with large datasets. At the same time, every step, parameter, and result was logged, so nothing was lost and experiments could be tracked easily.

For testing and validation, models were evaluated on completely unseen data to ensure fairness. We also checked how the system handles real-world problems like outliers and data drift. In addition, model explanations were verified to make sure the predictions were understandable and not just accurate.

Finally, stress testing was done by pushing the system to its limits. It was tested on low-resource devices, large datasets with over 100,000 rows, and even across multi-cloud environments. The system remained stable and performed well, showing that it is robust enough for practical use.

CHAPTER 9: RESULTS



Type '/' to search projects

[Help](#) [Docs](#) [Sponsors](#) [Log in](#) [Register](#)

adamops 0.1.1

`pip install adamops`

✓ Latest version

Released: Mar 13, 2026

A comprehensive MLOps library for end-to-end machine learning workflows

Navigation

[Project description](#)[Release history](#)[Download files](#)

Verified details

These details have been [verified by PyPI](#)

Maintainers



Abhishek05-mavrick

Project description



AdamOps

python 3.8+ license MIT version 0.1.0

AdamOps is a comprehensive MLOps library for end-to-end machine learning workflows. It provides a unified interface for data processing, model training, evaluation, deployment, and monitoring.

Features

Data Module (DataOps)

- Loaders: CSV, Excel, JSON, SQL, API, compressed files with auto-encoding detection
- Validators: Type validation, missing values, duplicates, shape, statistical checks

AdamOps

latest

Search docs

USER GUIDE

Getting Started

Installation

Using the Model Playground

Using AdamOps Studio

API REFERENCE

adamops

[/ Getting Started](#)[View page source](#)

Getting Started

Installation

You can install AdamOps and all of its extensive features directly from PyPI:

```
pip install adamops
```

Using the Model Playground

AdamOps features a built-in Streamlit dashboard for instantaneously visualizing your trained models, evaluating metrics, and generating predictions.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from adamops.deployment.playground import launch_playground

# 1. Load data
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. Train a standard model
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

# 3. Launch the playground to interact with your model!
launch_playground(model=model, X_test=X_test, y_test=y_test)
```

CHAPTER 10: ANALYSIS & CONCLUSION

ADAMOPS clearly shows one thing most ML problems are not about models, they are about execution. The system reduces that gap. Instead of spending hours setting up pipelines, cleaning data, tracking experiments, and deploying models, everything is handled in one flow. This directly cuts down human mistakes and saves time. The results support this. Faster pipeline creation, near same accuracy as manual models, and low latency make it practical, not just theoretical. But there are still gaps. Right now it works mainly on structured data, so it misses out on image and text-heavy use cases which are very common today. That limits its real-world reach. Also, full automation can sometimes hide important decisions, which might be risky in critical systems. Still, for startups, students, and small teams, this is a strong advantage. They don't have the resources to manage complex MLOps setups, and this system removes that barrier. The biggest strength is simplicity with power. You don't need deep expertise to get production-level output. Looking forward, adding support for LLMs, real-time data, and better monitoring will decide how far this can scale. If those are done right, ADAMOPS can move from a useful tool to a standard way of building ML systems.

REFERENCES

1. Zarour, Mohammad, Hamza Alzabut, and Khaled Alsarayrah. "MLOps best practices, challenges and maturity models: A systematic literature review." *Information and Software Technology* (2025).
2. Eken, Beyza, et al. "A multivocal review of MLOps practices, challenges and open issues." *ACM Computing Surveys* (2024).
3. Woźniak, Adrian P., Mateusz Milczarek, and Joanna Woźniak. "MLOps Components, Tools, Process and Metrics-A Systematic Literature Review." *IEEE Access* (2025).
4. Hanchuk, Danylo O., and Serhiy O. Semerikov. "Implementing MLOps practices for effective machine learning model deployment: A meta synthesis." *CEUR Workshop Proceedings*. 2025.
5. Bayram, Firas, and Bestoun S. Ahmed. "Towards trustworthy machine learning in production: An overview of the robustness in mlops approach." *ACM Computing Surveys* 57.5 (2025).
6. John, Meenu Mary, Helena Holmström Olsson, and Jan Bosch. "An empirical guide to MLOps adoption: Framework, maturity model and taxonomy." *Information and Software Technology* 183 (2025).
7. Li, Yutong, et al. "Maturity Framework for Operationalizing Machine Learning Applications in Health Care: Scoping Review." *Journal of Medical Internet Research* 27 (2025).
8. Asghar, Suleman, and Martins Amola. "Tools and Frameworks for MLOps." (2025).
9. Matthew, Bamidele. "Achieving Continuous Integration and Continuous Delivery (CI/CD) in MLOps with DataOps." (2025).
10. Manchana, Ramakrishna. "The power of convergence: platform ops as the unifying force for DevOps DataOps and MLOPs." *International Journal of Science and Research (IJSR)* 13.9 (2024)

APPENDIX: PLAGIARISM REPORT



3% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

-  **6 AI-generated only 3%**
Likely AI-generated text from a large-language model.
-  **0 AI-generated text that was AI-paraphrased 0%**
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



APPENDIX: PUBLICATION SUBMISSION PAGE



Acceptance Notification – ICMTIMS '26 Inbox x

ICMTIMS conference <icmtimsconference@gmail.com>
to vijji.c, me, Deepesh, aquibg4567, abhishek05abs

Fri, Mar 27, 1:39 PM (5 days ago) ☆ 😊 ↶ ⋮

Dear Author **Viji C, Ayush Mishra A, Deepesh Kumar Jha, Mohd Mustafa, Abhishek Virupakasha Ammanagi**

We are pleased to inform you that your paper titled “**ADAMOPS: An Automated End-to-End MLOps Framework**” with Paper ID: **ICMTIMS – 327** has been officially accepted for presentation at the **International Conference on Multidisciplinary Technology, Innovation, and Management Science (ICMTIMS '26)** after a successful peer-review process.

Date of Acceptance: 27.03.2026

Your paper has been evaluated by the Technical Review Committee and found suitable for publication and presentation. Kindly ensure that all reviewer-suggested corrections (if any) are incorporated into the camera-ready version before final submission. The official camera-ready template will be shared with you shortly.

Review Comments:

Comment 1:

The paper presents a comprehensive and well-structured MLOps framework (ADAMOPS) that effectively integrates DataOps, DevOps, and ML workflows into a unified automated pipeline. The modular architecture and inclusion of tools like MLflow, DVC, Docker, and Kubernetes demonstrate strong practical relevance and make the framework valuable for both beginners and practitioners.

Comment 2:

The methodology is clearly explained with detailed workflow stages, covering data ingestion, model development, testing, and deployment automation. The inclusion of monitoring, drift detection, and optimization aspects adds depth to the framework. With minor improvements in experimental validation and comparative benchmarking, the work is suitable for publication.

Important Notes:

- The final camera-ready paper must be submitted as per the official conference template before the specified deadline.
- ICMTIMS '26 proceedings are indexed in **Scopus-indexed outlets**.
- At least one author must register and present the paper at the conference for inclusion in the final proceedings.

Registration Instructions:

1. Complete your registration at <https://icmtims.com/Payment.php>
2. After payment, kindly fill out the Google Form with payment details <https://forms.gle/sNG1AT7QWYQmQbR8A>

APPENDIX: SOURCE CODE

```
"""
AdamOps Feature Engineering Module

Provides encoding, scaling, feature selection, and auto feature generation.
"""

from typing import Dict, List, Optional, Tuple, Union
import numpy as np
import pandas as pd
from sklearn.preprocessing import (
    OneHotEncoder, LabelEncoder, OrdinalEncoder, StandardScaler,
    MinMaxScaler, RobustScaler, MaxAbsScaler, PolynomialFeatures
)
from sklearn.feature_selection import (
    VarianceThreshold, SelectKBest, mutual_info_classif, mutual_info_regression, RFE
)
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from adamops.utils.logging import get_logger

logger = get_logger(__name__)

# =====
# Encoding
# =====

def encode_onehot(
    df: pd.DataFrame, columns: List[str], drop_first: bool = False,
    handle_unknown: str = "ignore"
) -> pd.DataFrame:
    """One-hot encode categorical columns."""
    df = df.copy()
    for col in columns:
        dummies = pd.get_dummies(df[col], prefix=col, drop_first=drop_first)
        df = pd.concat([df.drop(col, axis=1), dummies], axis=1)
    logger.info(f"One-hot encoded {len(columns)} columns")
    return df

def encode_label(df: pd.DataFrame, columns: List[str]) -> Tuple[pd.DataFrame, Dict]:
    """Label encode categorical columns. Returns df and encoders dict."""
    df = df.copy()
    encoders = {}
    for col in columns:
        le = LabelEncoder()
```

```

        df[col] = le.fit_transform(df[col].astype(str))
        encoders[col] = le
    logger.info(f"Label encoded {len(columns)} columns")
    return df, encoders

```

```

def encode_ordinal(
    df: pd.DataFrame, columns: List[str],
    categories: Optional[Dict[str, List]] = None
) -> pd.DataFrame:
    """Ordinal encode columns with optional category order."""
    df = df.copy()
    for col in columns:
        if categories and col in categories:
            cat_map = {v: i for i, v in enumerate(categories[col])}
            df[col] = df[col].map(cat_map)
        else:
            df[col] = pd.Categorical(df[col]).codes
    return df

```

```

def encode_target(
    df: pd.DataFrame, columns: List[str], target: str, smoothing: float = 1.0
) -> pd.DataFrame:
    """Target encode categorical columns."""
    df = df.copy()
    global_mean = df[target].mean()

    for col in columns:
        agg = df.groupby(col)[target].agg(['mean', 'count'])
        smooth = (agg['count'] * agg['mean'] + smoothing * global_mean) /
        (agg['count'] + smoothing)
        df[col + '_target'] = df[col].map(smooth)
        df = df.drop(col, axis=1)

    return df

```

```

def encode(
    df: pd.DataFrame, columns: List[str], method: str = "onehot", **kwargs
) -> pd.DataFrame:
    """Encode categorical columns with specified method."""
    if method == "onehot":
        return encode_onehot(df, columns, **kwargs)
    elif method == "label":
        return encode_label(df, columns, **kwargs)[0]
    elif method == "ordinal":
        return encode_ordinal(df, columns, **kwargs)
    elif method == "target" and "target" in kwargs:
        return encode_target(df, columns, kwargs["target"])
    else:
        raise ValueError(f"Unknown encoding method: {method}")

```

```

# =====
# Scaling
# =====

def scale_standard(df: pd.DataFrame, columns: Optional[List[str]] = None) ->
pd.DataFrame:
    """Standardize features (zero mean, unit variance)."""
    df = df.copy()
    cols = columns or df.select_dtypes(include=[np.number]).columns.tolist()
    scaler = StandardScaler()
    df[cols] = scaler.fit_transform(df[cols])
    return df

def scale_minmax(df: pd.DataFrame, columns: Optional[List[str]] = None) ->
pd.DataFrame:
    """Scale features to [0, 1] range."""
    df = df.copy()
    cols = columns or df.select_dtypes(include=[np.number]).columns.tolist()
    scaler = MinMaxScaler()
    df[cols] = scaler.fit_transform(df[cols])
    return df

def scale_robust(df: pd.DataFrame, columns: Optional[List[str]] = None) ->
pd.DataFrame:
    """Scale with median and IQR (robust to outliers)."""
    df = df.copy()
    cols = columns or df.select_dtypes(include=[np.number]).columns.tolist()
    scaler = RobustScaler()
    df[cols] = scaler.fit_transform(df[cols])
    return df

def scale(
    df: pd.DataFrame, method: str = "standard", columns: Optional[List[str]] = None
) -> pd.DataFrame:
    """Scale numeric columns with specified method."""
    if method == "standard":
        return scale_standard(df, columns)
    elif method == "minmax":
        return scale_minmax(df, columns)
    elif method == "robust":
        return scale_robust(df, columns)
    elif method == "maxabs":
        df = df.copy()
        cols = columns or df.select_dtypes(include=[np.number]).columns.tolist()
        df[cols] = MaxAbsScaler().fit_transform(df[cols])
        return df
    else:
        raise ValueError(f"Unknown scaling method: {method}")

# =====

```

```

# Feature Selection
# =====

def select_by_variance(
    df: pd.DataFrame, threshold: float = 0.0, columns: Optional[List[str]] = None
) -> pd.DataFrame:
    """Remove low variance features."""
    cols = columns or df.select_dtypes(include=[np.number]).columns.tolist()
    selector = VarianceThreshold(threshold=threshold)
    selected = selector.fit_transform(df[cols])
    selected_cols = [cols[i] for i in selector.get_support(indices=True)]
    df_result = df.drop(cols, axis=1)
    df_result[selected_cols] = selected
    logger.info(f"Selected {len(selected_cols)}/{len(cols)} features by variance")
    return df_result

def select_by_correlation(
    df: pd.DataFrame, threshold: float = 0.9, target: Optional[str] = None
) -> pd.DataFrame:
    """Remove highly correlated features."""
    df = df.copy()
    num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
    if target and target in num_cols:
        num_cols.remove(target)

    corr = df[num_cols].corr().abs()
    upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
    to_drop = [c for c in upper.columns if any(upper[c] > threshold)]

    logger.info(f"Dropping {len(to_drop)} highly correlated features")
    return df.drop(to_drop, axis=1)

def select_by_importance(
    df: pd.DataFrame, target: str, n_features: int = 10, task: str = "classification"
) -> pd.DataFrame:
    """Select features by tree-based importance."""
    X = df.drop(target, axis=1).select_dtypes(include=[np.number])
    y = df[target]

    model = RandomForestClassifier(n_estimators=50, random_state=42) if task ==
"classification" \
        else RandomForestRegressor(n_estimators=50, random_state=42)
    model.fit(X, y)

    importance = pd.Series(model.feature_importances_,
index=X.columns).sort_values(ascending=False)
    top_features = importance.head(n_features).index.tolist()

    logger.info(f"Selected top {n_features} features by importance")
    return df[[target] + top_features]

```

```

def select_features(
    df: pd.DataFrame, target: str, method: str = "importance", n_features: int = 10,
    **kwargs
) -> pd.DataFrame:
    """Select features using specified method."""
    if method == "variance":
        return select_by_variance(df, **kwargs)
    elif method == "correlation":
        return select_by_correlation(df, target=target, **kwargs)
    elif method == "importance":
        return select_by_importance(df, target, n_features, **kwargs)
    else:
        raise ValueError(f"Unknown selection method: {method}")

# =====
# Feature Generation
# =====

def generate_polynomial(
    df: pd.DataFrame, columns: List[str], degree: int = 2, include_bias: bool = False
) -> pd.DataFrame:
    """Generate polynomial features."""
    df = df.copy()
    poly = PolynomialFeatures(degree=degree, include_bias=include_bias)
    poly_features = poly.fit_transform(df[columns])
    poly_names = poly.get_feature_names_out(columns)
    df_poly = pd.DataFrame(poly_features, columns=poly_names, index=df.index)
    return pd.concat([df.drop(columns, axis=1), df_poly], axis=1)

def generate_interactions(
    df: pd.DataFrame, columns: List[str], operations: List[str] = ["multiply"]
) -> pd.DataFrame:
    """Generate interaction features between columns."""
    df = df.copy()
    for i, col1 in enumerate(columns):
        for col2 in columns[i+1:]:
            if "multiply" in operations:
                df[f"{col1}_x_{col2}"] = df[col1] * df[col2]
            if "add" in operations:
                df[f"{col1}+_ {col2}"] = df[col1] + df[col2]
            if "divide" in operations:
                df[f"{col1}_/_ {col2}"] = df[col1] / (df[col2] + 1e-8)
    return df

def generate_datetime_features(df: pd.DataFrame, column: str) -> pd.DataFrame:
    """Extract datetime features from a column."""
    df = df.copy()
    dt = pd.to_datetime(df[column])
    prefix = column
    df[f"{prefix}_year"] = dt.dt.year
    df[f"{prefix}_month"] = dt.dt.month

```

```

df[f"{prefix}_day"] = dt.dt.day
df[f"{prefix}_dayofweek"] = dt.dt.dayofweek
df[f"{prefix}_hour"] = dt.dt.hour
df[f"{prefix}_is_weekend"] = dt.dt.dayofweek.isin([5, 6]).astype(int)
return df

```

```

def auto_feature_engineering(
    df: pd.DataFrame, target: Optional[str] = None,
    polynomial: bool = False, interactions: bool = False, datetime_cols:
Optional[List[str]] = None
) -> pd.DataFrame:
    """Automatic feature engineering pipeline."""
    logger.info("Running auto feature engineering")

    num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
    if target and target in num_cols:
        num_cols.remove(target)

    if datetime_cols:
        for col in datetime_cols:
            df = generate_datetime_features(df, col)

    if polynomial and len(num_cols) <= 5:
        df = generate_polynomial(df, num_cols[:5], degree=2)

    if interactions and len(num_cols) >= 2:
        df = generate_interactions(df, num_cols[:4])

    logger.info(f"Feature engineering complete. New shape: {df.shape}")
    return df

```